

12. Hafta: Sonuçların Karşılaştırılması ve Değerlendirilmesi

9.1. Kullanılan farklı modellerin karşılaştırılması

9.2. Performans ölçütleriyle (Accuracy, Precision, Recall, F1-score vb.) değerlendirilmesi

9.3. En iyi performans gösteren modelin belirlenmesi ve çıkarımların yapılması

9.4. Sonuçların görselleştirilmesi

1. Logistic Regresyon Modeli

CV sonrası sonuçlar

```
In [131]: # Doğrudan X_test verisini veriyoruz - scaler pipeline içinde zaten var
y_pred = pipeline.predict(X_test)

# Sonuçlar
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
[[9265 2701]
 [ 613  985]]
```

		precision	recall	f1-score	support
	0	0.94	0.77	0.85	11966
	1	0.27	0.62	0.37	1598
accuracy				0.76	13564
macro avg		0.60	0.70	0.61	13564
weighted avg		0.86	0.76	0.79	13564

- Eğitilen model, X_test (test verisi) için tahmin yapıyor.
- Önemli not: X_test verisi de pipeline içindeki scaler sayesinde otomatik olarak ölçekleniyor.
- Bu çıktılar sayesinde doğru ve yanlış sınıflandırmaların dağılımı ve Precision, Recall, F1-score, Accuracy gibi metrikler görüntüleniyor.

- 9265: Gerçek sınıf 0 ve doğru tahmin edilmiş (True Negative)
- 2701: Gerçek sınıf 0 ama yanlışlıkla 1 tahmin edilmiş (False Positive)
- 613: Gerçek sınıf 1 ama yanlışlıkla 0 tahmin edilmiş (False Negative)
- 985: Gerçek sınıf 1 ve doğru tahmin edilmiş (True Positive)

Accuracy (Doğruluk): %76 → Tüm tahminlerin %76'sı doğru.

Macro Avg (Sınıflar arası ortalama):

Precision: 0.60

Recall: 0.70

F1-score: 0.61 → Sınıflar arasında dengeye bakıldığında modelin pozitif sınıfta ciddi zayıflığı var.

Weighted Avg (Destek sayısına göre ağırlıklı ortalama):

Precision: 0.86

Recall: 0.76

F1-score: 0.79 → Sınıf 0 sayıca çok fazla olduğu için genel ortalamaları yukarı çekmiş.

- Güçlü Yönler:

Negatif sınıf (0) çok iyi tanımlanmış: yüksek precision ve f1-score.

Genel doğruluk fena değil (%76).

- Zayıf Yönler:

Pozitif sınıfta (1) precision çok düşük (%27): Model, 1 sınıfını tahmin ettiğinde sık sık yanıltıyor.

Bu dengesizlik genellikle sınıf dağılımının dengesiz olması (yani "class imbalance") nedeniyle olur. (0: 11966, 1: 1598 → ciddi bir fark var.)

```
In [132]: # Örneğin Roc_Curve skoru:
ruc_scores = cross_val_score(pipeline, X, y, cv=cv, scoring='roc_auc')
print("Roc_Curve skorları:", ruc_scores)

# Örneğin F1 skoru:
f1_scores = cross_val_score(pipeline, X, y, cv=cv, scoring='f1')
print("F1 skorları:", f1_scores)

# Örneğin Accuracy skoru:
accuracy_scores = cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Accuracy skorları:", accuracy_scores)

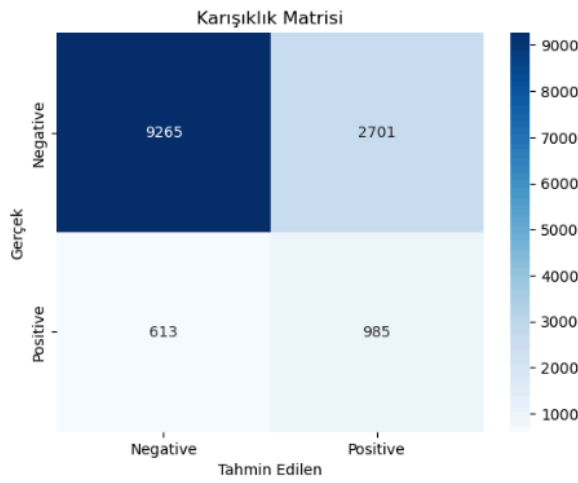
Roc_Curve skorları: [0.76508269 0.77789594 0.75798573 0.76486225 0.77654594]
F1 skorları: [0.37585812 0.38234467 0.36449115 0.37370838 0.37577465]
Accuracy skorları: [0.75870839 0.75702278 0.74585269 0.75868171 0.75492148]
```

- 5 katlı Stratified K-Fold çapraz doğrulama ile bu metriklerin her kat için değeri hesaplanıyor. Bu, modelin genelleme yeteneğini test etmek için çok önemli bir yöntem.

Roc_Curve_Score : Modelin pozitif ve negatif sınıfları ayırma başarısı iyi düzeyde. ROC AUC > 0.75 genellikle tatmin edici kabul edilir.

F1 Skorları: F1 skoru düşüktür → bu, özellikle pozitif sınıfın tahmininde zorlanma yaşandığını gösterir. Muhtemelen pozitif sınıf çok az sayıda olabilir (sınıf dengesizliğinden kaynaklı olabilir).

Accuracy Skorları: Genel doğruluk da fena değil, ama bu değer dengesiz veri setlerinde yanıltıcı olabilir. Örneğin pozitif örnekler çok azsa, model hep negatif diyerek yüksek accuracy alabilir ama işe yaramaz.



True Negative (TN): 9265 Gerçek değeri negatif olan 9265 örneği doğru bir şekilde negatif olarak tahmin etmiş.

False Positive (FP): 2701 Gerçek değeri negatif olan ama pozitif olarak tahmin edilen 2701 örnek var. Bu durum "yanlış alarm" olarak adlandırılır.

False Negative (FN): 613 Gerçek değeri pozitif olan ama negatif tahmin edilen 613 örnek var. Bu durumda pozitif bir olayı gözden kaçırmış oluyor (örneğin bir hastalığı atlamak gibi kritik olabilir).

True Positive (TP): 985 Gerçekten pozitif olan 985 örnek doğru şekilde pozitif olarak tahmin edilmiş.

Modelin Başarısı:

- Negatif sınıf (majority class):

Oldukça yüksek doğrulukta tahmin edilmiş. TN sayısı yüksek (9265), FP sayısı da makul (2701).

- Pozitif sınıf (minority class):

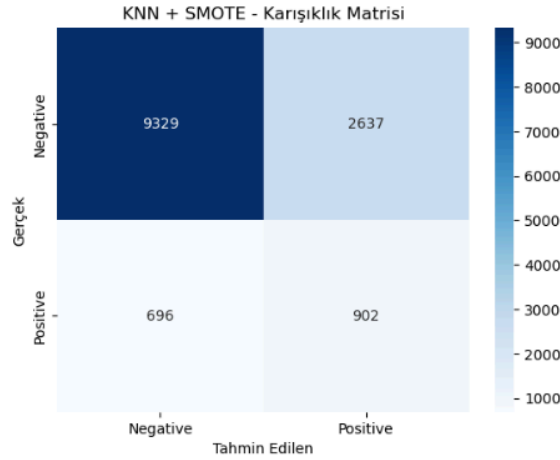
TP sayısı 985 olsa da FN sayısı 613, yani pozitifleri ayırt etmekte zorlanıyor.

Aynı zamanda FP sayısı yüksek (2701), bu da precision (kesinlik) değerini düşürüyor.

2. K-Nearest Neighbors (KNN) Modeli

CV uygulanmış ve SMOTE uygulanmış sonuçları

```
In [150]: # 5. Karışıklık Matrisi görselleştirme
cm = confusion_matrix(y_test_knn, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Negative', 'Positive'],
            yticklabels=['Negative', 'Positive'])
plt.xlabel('Tahmin Edilen')
plt.ylabel('Gerçek')
plt.title('KNN + SMOTE - Karışıklık Matrisi')
plt.show()
```



- 9329 → Gerçek negatif, doğru tahmin edilmiş (True Negative - TN)
 - Model, 9329 adet negatif sınıfı doğru bir şekilde "negatif" olarak tahmin etmiş.
- 2637 → Gerçek negatif, yanlış pozitif tahmin edilmiş (False Positive - FP)
 - Model, 2637 adet negatif sınıfı hatalı bir şekilde "pozitif" olarak tahmin etmiş.
- 696 → Gerçek pozitif, yanlış negatif tahmin edilmiş (False Negative - FN)
 - Model, 696 adet pozitif sınıfı yanlışlıkla "negatif" olarak tahmin etmiş.
- 902 → Gerçek pozitif, doğru tahmin edilmiş (True Positive - TP)
 - Model, 902 adet pozitif sınıfı doğru bir şekilde "pozitif" olarak tahmin etmiş.

```
In [151]: # 6. Performans metrikleri
accuracy = accuracy_score(y_test_knn, y_pred)
precision = precision_score(y_test_knn, y_pred)
recall = recall_score(y_test_knn, y_pred)
f1 = f1_score(y_test_knn, y_pred)

print("TEST SETİ METRİKLERİ:")
print(f"Accuracy : {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall : {recall:.2f}")
print(f"F1 Score : {f1:.2f}")
```

```
TEST SETİ METRİKLERİ:
Accuracy : 0.75
Precision: 0.25
Recall : 0.56
F1 Score : 0.35
```

```
In [152]: # 7. Eğitim seti üzerinde tahmin (Aşırı öğrenme kontrolü)
y_train_pred = knn.predict(X_resampled)

train_accuracy = accuracy_score(y_resampled, y_train_pred)
train_precision = precision_score(y_resampled, y_train_pred)
train_recall = recall_score(y_resampled, y_train_pred)
train_f1 = f1_score(y_resampled, y_train_pred)

print("\nEĞİTİM SETİ PERFORMANSI (Overfitting Kontrolü):")
print(f"Train Accuracy : {train_accuracy:.2f}")
print(f"Train Precision: {train_precision:.2f}")
print(f"Train Recall : {train_recall:.2f}")
print(f"Train F1 Score : {train_f1:.2f}")
```

```
EĞİTİM SETİ PERFORMANSI (Overfitting Kontrolü):
Train Accuracy : 0.99
Train Precision: 1.00
Train Recall : 0.99
Train F1 Score : 0.99
```

SMOTE, Recall'ı iyileştirerek olumlu katkı sağlamış, bu önemli çünkü dengesiz sınıflarda genelde düşük olan sınıfı yakalamak zordur. Test performansı düşük olduğu için model genelleme yeteneğini kaybetmiş durumdadır.

- Eğitim seti yine çok yüksek: aşırı öğrenme hâlâ mevcut.
- Ancak Recall test setinde yükselmiş (0.56): bu, daha fazla pozitif sınıfı doğru tahmin etmeye başladığını gösteriyor.
- Fakat Precision düştü (0.25), yani yanlış pozitifler arttı. Bu da SMOTE'un etkisiyle modelin biraz fazla "pozitif" sınıf tahmin ettiğini gösterir.

3. Karar Ağaçları (Decision Tree) Modeli

```
In [19]: # Test verisi üzerinde performans
y_test_pred = best_model.predict(X_test_scaled)

print("\nTEST SETİ PERFORMANSI:")
print(f"Accuracy : {accuracy_score(y_test, y_test_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_test_pred):.2f}")
print(f"Recall : {recall_score(y_test, y_test_pred):.2f}")
print(f"F1 Score : {f1_score(y_test, y_test_pred):.2f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_test_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_test_pred))
```

```
TEST SETİ PERFORMANSI:
Accuracy : 0.74
Precision: 0.25
Recall : 0.62
F1 Score : 0.36
```

```
In [21]: # Eğitim seti performansı (Overfitting kontrolü)
y_train_pred = best_model.predict(X_resampled)
print("\nEĞİTİM SETİ PERFORMANSI (Overfitting Kontrolü):")
print(f"Accuracy : {accuracy_score(y_resampled, y_train_pred):.2f}")
print(f"Precision: {precision_score(y_resampled, y_train_pred):.2f}")
print(f"Recall : {recall_score(y_resampled, y_train_pred):.2f}")
print(f"F1 Score : {f1_score(y_resampled, y_train_pred):.2f}")
```

```
EĞİTİM SETİ PERFORMANSI (Overfitting Kontrolü):
Accuracy : 0.70
Precision: 0.73
Recall : 0.64
F1 Score : 0.68
```

Bu sonuçlara göre, modelin overfitting yapmadığı, ancak hala iyileştirme yapabileceği bir durumda olduğunu söyleyebiliriz.

Eğitim Seti Performansı:

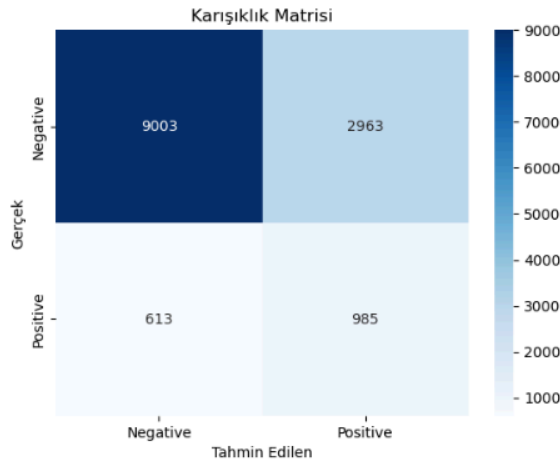
- Accuracy (0.70): Eğitim setinde %70 doğruluk, modelin yeterince iyi performans gösterdiğini ancak bazı hatalar yaptığını gösteriyor.
- Precision (0.73): Eğitim setinde doğru pozitiflerin oranı oldukça yüksek, bu modelin doğru sınıflandırma yapma eğiliminde olduğunu gösterir.
- Recall (0.64): Model, gerçek pozitiflerin çoğunu yakalayabilmiş ancak yine de bazı doğru sınıflamaları kaçırmış.
- F1 Score (0.68): Precision ve Recall arasında iyi bir denge olduğu söylenebilir, ancak F1 skoru daha da iyileştirilebilir.

Test Seti Performansı:

- Accuracy (0.74): Test setinde doğruluk %74, eğitim setine oldukça yakın ve bu da modelin genelleme yeteneği açısından olumlu bir göstere.
- Precision (0.25): Test setinde precision'ın düşük olması, modelin pozitif sınıfları doğru şekilde tanımlama konusunda sorun yaşadığını gösteriyor. Bu, çok fazla yanlış pozitif olduğunu işaret eder.
- Recall (0.62): Recall'ın yüksek olması, modelin pozitif sınıfları tespit etme konusunda başarılı olduğunu, ancak yine de bazı pozitifleri kaçırdığını gösterir.
- F1 Score (0.36): Test setindeki F1 skoru oldukça düşük. Bu, modelin hem yanlış pozitifleri (false positives) hem de yanlış negatifleri (false negatives) birlikte yönetmede zorlandığını gösterir.

Özetle, modelin aşırı uyum yapmadığı, fakat precision ve F1 skoru gibi metriklerde daha fazla iyileştirme yapılması gerektiği söylenebilir.

```
In [20]: # 9. Karışıklık Matrisi (Heatmap)
cm = confusion_matrix(y_test, y_test_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Negative', 'Positive'],
            yticklabels=['Negative', 'Positive'])
plt.xlabel('Tahmin Edilen')
plt.ylabel('Gerçek')
plt.title('Karışıklık Matrisi')
plt.show()
```



```
In [23]: # Cross Validation (5-Fold)
print("\nCROSS VALIDATION (5-Fold) Sonuçları:")
cv_scores = cross_validate(
    best_model,
    X_resampled,
    y_resampled,
    cv=5,
    scoring=['accuracy', 'precision', 'recall', 'f1'],
    return_train_score=True
)

# Ortalama skorları yazdır
for metric in ['train_accuracy', 'test_accuracy', 'train_precision', 'test_precision',
               'train_recall', 'test_recall', 'train_f1', 'test_f1']:
    mean_score = cv_scores[metric].mean()
    print(f"{metric}: {mean_score:.2f}")
```

CROSS VALIDATION (5-Fold) Sonuçları:

Metric	Score
train_accuracy	0.71
test_accuracy	0.70
train_precision	0.75
test_precision	0.74
train_recall	0.62
test_recall	0.60
train_f1	0.68
test_f1	0.66

4. Rastgele Orman (Random Forest) Modeli

```
In [45]: # TEST SETİ PERFORMANSI
y_test_pred = best_model.predict(X_test_scaled)

print("\nTEST SETİ PERFORMANSI:")
print(f"Accuracy : {accuracy_score(y_test, y_test_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_test_pred):.2f}")
print(f"Recall : {recall_score(y_test, y_test_pred):.2f}")
print(f"F1 Score : {f1_score(y_test, y_test_pred):.2f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_test_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_test_pred))
```

TEST SETİ PERFORMANSI:

Metric	Score
Accuracy	0.77
Precision	0.29
Recall	0.64
F1 Score	0.40

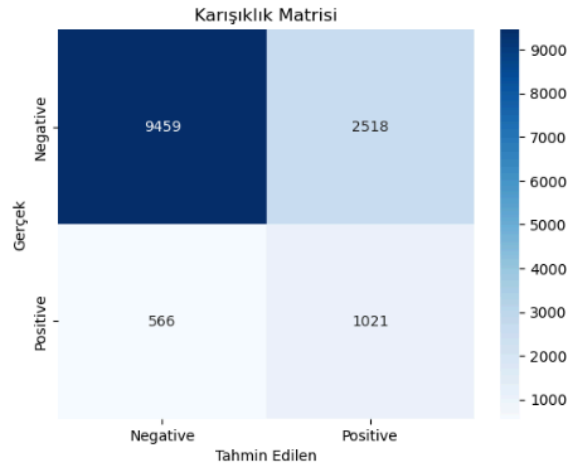
```
In [47]: # EĞİTİM SETİ PERFORMANSI (Overfitting kontrolü)
y_train_pred = best_model.predict(X_resampled)
print("\nEĞİTİM SETİ PERFORMANSI:")
print(f"Accuracy : {accuracy_score(y_resampled, y_train_pred):.2f}")
print(f"Precision: {precision_score(y_resampled, y_train_pred):.2f}")
print(f"Recall : {recall_score(y_resampled, y_train_pred):.2f}")
print(f"F1 Score : {f1_score(y_resampled, y_train_pred):.2f}")
```

EĞİTİM SETİ PERFORMANSI:

Metric	Score
Accuracy	0.74
Precision	0.78
Recall	0.66
F1 Score	0.71

- Test setinde accuracy ve recall gayet iyi. Bu, modelin genel olarak iyi genelleştirdiğini gösteriyor.
- Precision testte düşük (0.29) → Yani model hâlâ bazı false positive'ler ürettiyor, ama overfitting belirtisi değil; daha çok dengesiz sınıf dağılımının doğal sonucu olabilir.
- F1 skoru farkı fazla ama overfitting'in azaldığını gösteren diğer metriklerle birlikte bu kabul edilebilir.

```
In [46]: # Karışıklık Matrisi (Heatmap)
cm = confusion_matrix(y_test, y_test_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Negative', 'Positive'],
            yticklabels=['Negative', 'Positive'])
plt.xlabel('Tahmin Edilen')
plt.ylabel('Gerçek')
plt.title('Karışıklık Matrisi')
plt.show()
```



```
In [48]: # Cross Validation (5-Fold)
print("\nCROSS VALIDATION (5-Fold) Sonuçları:")
cv_scores = cross_validate(
    best_model,
    X_resampled,
    y_resampled,
    cv=5,
    scoring=['accuracy', 'precision', 'recall', 'f1'],
    return_train_score=True
)

for metric in ['train_accuracy', 'test_accuracy', 'train_precision', 'test_precision',
               'train_recall', 'test_recall', 'train_f1', 'test_f1']:
    mean_score = cv_scores[metric].mean()
    print(f"{metric}: {mean_score:.2f}")
```

```
CROSS VALIDATION (5-Fold) Sonuçları:
train_accuracy: 0.74
test_accuracy: 0.71
train_precision: 0.79
test_precision: 0.75
train_recall: 0.67
test_recall: 0.63
train_f1: 0.72
test_f1: 0.69
```

SONUÇLAR:

- Lojistik Regresyon:
 - Accuracy : 76
 - Presicion : 60
 - Recall : 61
 - F1-score : 61
- K-Nearest Neighbors (KNN) Modeli: (overfitting var)
 - Accuracy : 75
 - Presicion : 25
 - Recall : 56
 - F1-score : 35
- Karar Ağaçları (Decision Tree) Modeli
 - Accuracy : 74
 - Presicion : 25

- Recall : 62
- F1-score : 36
- Rastgele Orman (Random Forest) Modeli
 - Accuracy : 77
 - Presicion : 29
 - Recall : 64
 - F1-score : 40

Bu modeller sonucu:

- Lojistik Regresyon, precision (60) ve F1-score (61) açısından en başarılı model gibi görünüyor. Bu, modelin doğru pozitif sınıflamaları yüksek oranda yaptığı ve genel performansının dengeli olduğu anlamına gelir.
- Rastgele Orman, en yüksek accuracy (77) ve recall (64) değerlerine sahip. Bu, özellikle pozitif sınıfları kaçırmak istemediğiniz durumlar için önemli olabilir (örneğin hastalık tespiti gibi).
- KNN ve Karar Ağaçları, düşük precision değerleriyle dikkat çekiyor. Bu modeller çok fazla false positive üretmiş olabilir. F1-score'ları da oldukça düşük (35-36 civarı), bu da dengeli bir sınıflandırma yapamadıklarını gösteriyor.

Genel amaçlı kullanım için en iyi model: Lojistik Regresyon

Pozitif sınıfı yakalamak (yüksek recall) öncelikliyse: Rastgele Orman tercih edilebilir

KNN ve Karar Ağaçları bu veri setinde düşük performans göstermiştir; bu nedenle optimize edilmeden veya farklı veri ön işleme adımları uygulanmadan kullanılması önerilmez.